

Program 1: 1D Array: Insertion, Deletion and Traversal

Aim

To write a C program that performs insertion, deletion and traversal operations on a one-dimensional array.

Theory

An **array** is a collection of elements of the same data type stored in contiguous memory locations, accessed using an index (starting from 0 in C).

Traversal means visiting every element exactly once, usually to print or process it. Time complexity: $O(n)$.

Insertion at a given position requires shifting all elements from that position one place to the right to make room, then storing the new value. Time complexity: $O(n)$ ($O(1)$ when inserting at the end).

Deletion from a given position requires shifting all following elements one place to the left to fill the gap, then reducing the size by one. Time complexity: $O(n)$.

Algorithm

1. Read the size n and the n elements of the array.
2. Insert: check that the position is valid (1 to $n+1$) and the array is not full; shift elements right from the last index down to the position; store the value; $n = n + 1$.
3. Delete: check that the position is valid (1 to n); shift elements left from the position to the end; $n = n - 1$.
4. Traverse: loop i from 0 to $n-1$ and print $a[i]$.
5. Repeat using a menu until the user chooses Exit.

C Source Code

```
/*
 * Program : 1D Array - Insertion, Deletion and Traversal
 *
 * THEORY:
 * - An array stores elements of the same type in contiguous memory.
 * - Traversal : visit every element once ->  $O(n)$ 
 * - Insertion : shift elements right from the position, then place the
 *               new value ->  $O(n)$ 
 * - Deletion : shift elements left over the position ->  $O(n)$ 
 * - Positions are entered 1-based (1 = first element).
 */
#include <stdio.h>
#define MAX 100

void traverse(int a[], int n) {
    if (n == 0) { printf("Array is empty.\n"); return; }
    printf("Array elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int insertElement(int a[], int n, int pos, int val) {
    if (n >= MAX) { printf("Array is full!\n"); return n; }
    if (pos < 1 || pos > n + 1) { printf("Invalid position!\n"); return n; }
```

```

        for (int i = n - 1; i >= pos - 1; i--)    /* shift right */
            a[i + 1] = a[i];
        a[pos - 1] = val;
        printf("Inserted %d at position %d.\n", val, pos);
        return n + 1;
    }

    int deleteElement(int a[], int n, int pos) {
        if (n == 0)                { printf("Array is empty!\n"); return n; }
        if (pos < 1 || pos > n) { printf("Invalid position!\n"); return n; }
        int removed = a[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)    /* shift left */
            a[i] = a[i + 1];
        printf("Deleted %d from position %d.\n", removed, pos);
        return n - 1;
    }

    int main() {
        int a[MAX], n, choice, pos, val;

        printf("Enter number of elements: ");
        scanf("%d", &n);
        printf("Enter %d elements: ", n);
        for (int i = 0; i < n; i++)
            scanf("%d", &a[i]);

        do {
            printf("\n1.Insert  2.Delete  3.Traverse  4.Exit\nEnter choice: ");
            scanf("%d", &choice);
            switch (choice) {
                case 1:
                    printf("Enter position and value: ");
                    scanf("%d %d", &pos, &val);
                    n = insertElement(a, n, pos, val);
                    break;
                case 2:
                    printf("Enter position to delete: ");
                    scanf("%d", &pos);
                    n = deleteElement(a, n, pos);
                    break;
                case 3:
                    traverse(a, n);
                    break;
                case 4:
                    printf("Exiting...\n");
                    break;
                default:
                    printf("Invalid choice!\n");
            }
        } while (choice != 4);
        return 0;
    }
}

```

Sample Input

Input: 5 elements (10 20 30 40 50) -> Insert 25 at position 3 -> Traverse -> Delete position 2 -> Traverse -> Exit.

Sample Output (program run with the input above)

```

Enter number of elements: Enter 5 elements:
1.Insert  2.Delete  3.Traverse  4.Exit
Enter choice: Enter position and value: Inserted 25 at position 3.

1.Insert  2.Delete  3.Traverse  4.Exit
Enter choice: Array elements: 10 20 25 30 40 50

1.Insert  2.Delete  3.Traverse  4.Exit
Enter choice: Enter position to delete: Deleted 20 from position 2.

```

```
1.Insert 2.Delete 3.Traverse 4.Exit  
Enter choice: Array elements: 10 25 30 40 50
```

```
1.Insert 2.Delete 3.Traverse 4.Exit  
Enter choice: Exiting...
```